ITA0615 – MACHINE LEARNING

ASSIGNMENT 3

By:

Harshith G

192525394

Q1 - BAYES THEOREM

1. Let F = faulty, + = positive test. Given: P(F) = 0.03, P(+ | F) = 0.95, P(+ | ¬F) = 0.08.

$$P(F \mid +) = P(+|F)P(F) / [P(+|F)P(F) + P(+|\neg F)P(\neg F)] = (0.95 \times 0.03) / (0.95 \times 0.03 + 0.08 \times 0.97)$$

$$= 0.0285 / (0.0285 + 0.0776) = 0.0285 / 0.1061 = 0.2686 \text{ (about 26.9 \%)}$$

Two independent positive tests. The likelihoods multiply (tests are conditionally independent given the bearing's state):

$$P(F \mid +,+) = (0.95^2 \times 0.03) / (0.95^2 \times 0.03 + 0.08^2 \times 0.97) = 0.027075 / (0.027075 + 0.006208) = 0.8135 \text{ (about 81.4 \%)}$$

The same result is obtained sequentially: using 0.2686 as the new prior, $(0.95 \times 0.2686) / (0.95 \times 0.2686 + 0.08 \times 0.7314) = 0.2552 / 0.3137 = 0.8135$.

Role of the prior (base rate). In odds form, posterior odds = prior odds × likelihood ratio = $(0.03/0.97) \times (0.95/0.08) = 0.0309 \times 11.875 = 0.367$, which is a probability of 0.2686. The test is quite informative (LR ≈ 11.9) but the fault is rare, so the 8 % false-positive rate applied to the 97 % healthy bearings (7.76 % of all bearings) outweighs the true positives (2.85 % of all bearings): roughly three of every four single positives are false alarms. The same test with different base rates gives very different answers:

Prior P(F)	**P(F | one +)**	**Comment**
0.003	0.0345	very rare fault – positive is mostly a false alarm
0.03	0.2686	given case
0.30	0.8358	common fault – positive is convincing

2. Find-S and Candidate-Elimination on D1–D6

Attribute order is <Temperature, Load, Vibration>; ∅ is the most specific (rejects everything) and ? accepts any value. Find-S uses only positive examples; Candidate-Elimination also uses the negatives to specialise G.

Step	**Example**	**Label**	**Find-S: S**	**Candidate-Elimination: S | G**
0	–	–	<∅, ∅, ∅>	S = <∅, ∅, ∅> | G = {<?, ?, ?>}
1	D1	Yes	<High, Heavy, High>	S = <High, Heavy, High> | G = {<?, ?, ?>}

Step	Example	Label	Find-S: S	Candidate-Elimination: S | G
2	D2	Yes	<High, Heavy, ?>	S = <High, Heavy, ?> | G = {<?, ?, ?>}
3	D3	No	<High, Heavy, ?> (no change)	S = <High, Heavy, ?> | G = {<High, ?, ?>, <?, Heavy, ?>}
4	D4	No	<High, Heavy, ?> (no change)	S = <High, Heavy, ?> | G = {<?, Heavy, ?>}
5	D5	Yes	<?, Heavy, ?>	S = <?, Heavy, ?> | G = {<?, Heavy, ?>}
6	D6	No	<?, Heavy, ?> (no change)	S = <?, Heavy, ?> | G = {<?, Heavy, ?>}

Notes on the trace. At D3 (Normal, Light, Low) the minimal specialisations of <?,?,?> are <High,?,?>, <?,Heavy,?> and <?,?,High>; <?,?,High> is discarded because it is not more general than S. At D4 (High, Light, High) <High,?,?> wrongly covers the negative, so it is specialised to <High,Heavy,?> and <High,?,Low>; the first is less general than <?,Heavy,?> and the second is not more general than S, so both are dropped. After D5 the boundaries meet: **S = G = {<?, Heavy, ?>}**, i.e. the concept "Fails Early" is exactly "Load = Heavy". Find-S reaches the same hypothesis but gives no indication that it is unique, whereas S = G proves it.

3. Version space for D1–D3 and brute-force MAP

A hypothesis must cover D1 and D2 (so Vibration must be? because D1 is High and D2 is Low, Temperature ∈ {High, ?}, Load ∈ {Heavy, ?}) and must not cover D3 (Normal, Light, Low), which excludes <?, ?, ?>. Enumerating all 27 conjunctive hypotheses by brute force leaves exactly three consistent ones, which agree with the boundaries S = <High, Heavy,?> and G = {<High,?,?>, <?,Heavy,?>}.

Hypothesis	Description	P(D | h)	Uniform prior P(h)	Posterior P(h | D)
h1	<High, Heavy, ?>	1	1/3	1/3 = 0.3333
h2	<High, ?, ?>	1	1/3	1/3 = 0.3333
h3	<?, Heavy, ?>	1	1/3	1/3 = 0.3333

$P(h \mid D) = P(D \mid h)\,P(h) / \Sigma\, P(D \mid h')\,P(h') = (1 \times 1/3) / (3 \times 1/3) = 1/3$ for every h in the version space; 0 for all others.

With a uniform prior and noise-free data, every consistent hypothesis is equally probable, so MAP cannot choose between them.

4. Non-uniform prior, h_ML and h_MAP

Prior chosen: P(h1) = 0.25, P(h2) = 0.45, P(h3) = 0.30 (sums to 1). **Justification:** (i) Occam's preference for general hypotheses – h2 and h3 have two wildcards while h1 has one, so together they receive 0.75 of the mass; (ii) between the two general hypotheses, temperature is the physically most plausible driver of early bearing failure, so h2 gets slightly more than h3. Because the likelihood is 1 for all three, the posterior equals the normalised prior:

Hypothesis	Description	Prior	P(D | h)	Posterior P(h | D)
h1	<High, Heavy, ?>	0.25	1	0.25
h2	<High, ?, ?>	0.45	1	0.45
h3	<?, Heavy, ?>	0.30	1	0.30

h_ML: all three hypotheses have P(D | h) = 1, so the maximum-likelihood hypothesis is a three-way tie. h_MAP = h2 = <High, ?, ?> (posterior 0.45).

Reasoning Inquiry

When do h_ML and h_MAP coincide? (a) When the prior is uniform over the hypotheses (P(h) constant), because argmax P(D|h)P(h) = argmax P(D|h); (b) when the hypothesis with the highest likelihood also has the highest prior; (c) asymptotically, when the data are plentiful, because the likelihood term shrinks exponentially with every extra example while the prior stays fixed.

A strong prior changing the selection. Use all six records with a noise rate ε = 0.1, so P(D | h) = (1−ε)^(6−k) ε^k where k is the number of misclassified records. Counting errors on D1–D6: h1 makes 1 error (D5), h2 makes 2 errors (D4, D5), h3 makes 0 errors.

Hypothesis	Errors k	P(D | h)	Posterior, uniform prior	Posterior, strong prior (0.01 / 0.98 / 0.01)
h1 <High,Heavy,?>	1	$0.9^5 \times 0.1 = 0.05905$	0.0989	0.0479
h2 <High,?,?>	2	$0.9^4 \times 0.1^2 = 0.00656$	0.0110	0.5213
h3 <?,Heavy,?>	0	$0.9^6 = 0.53144$	0.8901	0.4309

Maximum likelihood is h3 under every prior. With a uniform prior h_MAP = h3 = h_ML. With the strong prior (0.98 on h2) the unnormalised scores are 0.01 × 0.05905 = 0.00059, 0.98 × 0.00656 =

0.00643 and 0.01 × 0.53144 = 0.00531; normalising by 0.01233 gives the posteriors above, so h_MAP switches to h2 although h_ML is still h3. The prior has overridden the evidence because the data (only six noisy records) were not strong enough to overcome it.

Why a single h_MAP can disagree with the whole version space. h_MAP discards the posterior mass of all other hypotheses. Take the new bearing x = (High, Light, Low): h1 predicts No (Load is not Heavy), h2 predicts Yes (Temperature is High) and h3 predicts No. With the prior above, h_MAP = h2 classifies x as **Yes**, but the posterior-weighted vote is P(Yes) = 0.45 and P(No) = 0.25 + 0.30 = 0.55, so the combined opinion of the version space says **No**. (With a uniform prior the vote is 1/3 Yes versus 2/3 No.) The weighted vote is the Bayes optimal classification; h_MAP is only the single most probable hypothesis, and the most probable hypothesis need not predict the most probable label.

Implementation*:* all numbers above were produced by the hand-written Python script q1.py (Bayes posterior, Find-S, Candidate-Elimination, version-space enumeration, MAP/ML), supplied with this report.

Q2 – COMPARE THE MDL PRINCIPLE, BAYES OPTIMAL CLASSIFIER AND GIBBS ALGORITHM

1. Description length of the two trees (12 bits per node, log2(200) = 7.6439 bits per error)

Tree	L(h) = nodes × 12	L(D | h) = errors × 7.6439	L(h) + L(D | h)	MDL choice
A (6 nodes, 8 errors)	72.00	61.15	133.15	**Tree A**
B (25 nodes, 1 error)	300.00	7.64	307.64	–

Tree A needs 174.5 fewer bits in total, so MDL prefers the smaller Tree A: the 53.5 bits saved by Tree B's seven fewer errors are far smaller than the 228 extra bits needed to describe its 19 additional nodes.

2. Node cost at which the preference reverses

Let c be the cost per node. Preference reverses when 6c + 8 log2(200) = 25c + 1 × log2(200):

$$19c = 7 \times 7.6439 = 53.51 \implies c^* = 2.816 \text{ bits per node}$$

For c < 2.82 bits per node Tree B has the shorter description (e.g. c = 2: A = 73.15, B = 57.64); at c = 2.816 both are 78.05 bits; for c > 2.82 (e.g. c = 3: A = 79.15, B = 82.64) Tree A wins. At the given 12 bits the preference for A is very robust – a node would have to cost less than a quarter of the assumed amount.

MDL and MAP. MAP selects h_MAP = argmax P(D | h) P(h) = argmin [−log2 P(D | h) − log2 P(h)]. By Shannon's coding theorem the optimal code length for an event of probability p is −log2 p bits. Hence L(h) = −log2 P(h) is the length of the optimal code for the hypothesis (the tree description, which encodes the prior: simpler trees have shorter codes, hence higher prior probability 2^−L(h)), and L(D | h) = −log2 P(D | h) is the optimal code length of the data given the hypothesis (only the exceptions need to be sent). Minimising L(h) + L(D | h) is therefore the same as maximising the posterior, provided the encodings are optimal for the corresponding priors and likelihoods.

3. MAP versus Bayes optimal classification of X

Class	Hypotheses voting	Σ P(h | D)	Result
+	h1, h4	0.30 + 0.15 = 0.45	
−	h2, h3, h5	0.25 + 0.20 + 0.10 = 0.55	**− wins**

- **(i) MAP hypothesis:** h1 has the highest posterior (0.30), and it predicts **+**.
- **(ii) Bayes optimal classifier:** argmax over classes of Σ P(h | D) · [h predicts class] gives **−** (0.55 versus 0.45).

The two disagree: the single most probable hypothesis votes + but the combined evidence of the other four hypotheses favours −.

4. Gibbs algorithm, expected errors and their relationship

Gibbs algorithm: (1) draw one hypothesis h at random from the current posterior P(h | D); (2) classify the new instance with this h alone. It needs one sampled hypothesis per prediction rather than a weighted vote over all of H.

Let p = P(true label = +) = 0.45 under the posterior. If the true hypothesis is drawn from the posterior, the true label of X is + with probability 0.45 and − with probability 0.55. The Gibbs prediction is an independent draw from the same distribution:

$$E[\text{error_Gibbs}] = P(+)\cdot P(-) + P(-)\cdot P(+) = 2 \times 0.45 \times 0.55 = 0.495$$

$$E[\text{error_BO}] = \min(0.45, 0.55) = 0.45$$ (BO predicts −, wrong whenever the true label is +)

Relationship. Haussler, Kearns and Schapire showed that $E[\text{error_Gibbs}] \leq 2 \cdot E[\text{error_BO}]$. Here $0.495 \leq 0.90$, which holds, and also $E[\text{error_BO}] \leq E[\text{error_Gibbs}]$ ($0.45 \leq 0.495$), as the Bayes optimal classifier is the best possible on average. The factor 2 bound is tight only when the posterior is lopsided (e.g. 0.99 / 0.01: Gibbs 0.0198 versus BO 0.01); here the posterior is nearly balanced, so the two errors are close.

5. Monte Carlo verification

Quantity	Analytical	Simulation	Absolute difference
Gibbs error	0.4950	0.4940	0.0010
Bayes optimal error	0.4500	0.4507	0.0007

In each draw, a hypothesis is sampled as the "true" one and an independent hypothesis is sampled for the Gibbs prediction; the error is the fraction of draws in which their labels on X differ. Both estimates agree with the analytical values to within sampling error (standard error ≈ 0.0011).

Design Inquiry

Why Bayes optimal may not equal any single hypothesis. The Bayes optimal classifier is a posterior-weighted vote over all hypotheses, so the function it computes is a combination of hypotheses and can lie outside H. Example: with H = {h1, h2, h3}, equal posteriors and predictions on instances (x1, x2,

x3) of h1 = (+, +, −), h2 = (+, −, +), h3 = (−, +, +), the weighted vote gives (+, +, +), a labelling that none of the three hypotheses produces.

Why it is rarely used directly. It must evaluate P(h | D) and the prediction of every hypothesis for each new instance, which costs time proportional to |H|. For decision trees, conjunctions or neural networks, |H| is exponential or infinite (e.g. $3^n + 1$ conjunctions for n features), and computing the exact posterior of each hypothesis is also intractable.

Practical advantage of Gibbs. It samples a single hypothesis and applies it, so the cost per prediction is that of one hypothesis, and sampling from the posterior can be done approximately (e.g. by MCMC) without enumerating H. Its expected error is at most twice that of the Bayes optimal classifier, which makes it a cheap approximation with a theoretical guarantee.

Implementation: MDL, weighted-vote and Gibbs calculations and the Monte Carlo simulation are in the supplied script q2.py (NumPy default_rng(42), 200,000 draws).

Q3 – BUILD AND EVALUATE A NAÏVE BAYES CLASSIFIER

1. Naïve Bayes assumption, decision rule, priors and conditional probability tables

Assumption: the features are conditionally independent given the class: P(x1, x2, x3 | c) = P(Income | c) · P(Employment | c) · P(Credit | c).

Decision rule:

$$\hat{c} = \text{argmax}_c \, P(c) \prod_j P(x_j \mid c) = \text{argmax}_c \, [\log P(c) + \Sigma_j \log P(x_j \mid c)]$$

Class priors (5 Yes and 5 No among 10 records): P(Yes) = 5/10 = 0.5, P(No) = 5/10 = 0.5.

Conditional probability tables

Feature value	P(· | Yes)	P(· | No)	Count Yes	Count No
Income = High	2/5 = 0.4	1/5 = 0.2	2	1
Income = Medium	2/5 = 0.4	1/5 = 0.2	2	1
Income = Low	1/5 = 0.2	3/5 = 0.6	1	3
Employment = Salaried	4/5 = 0.8	1/5 = 0.2	4	1
Employment = Self-employed	1/5 = 0.2	4/5 = 0.8	1	4
Credit = Good	4/5 = 0.8	1/5 = 0.2	4	1
Credit = Poor	1/5 = 0.2	4/5 = 0.8	1	4

2. Manual classification

Application (Low, Self-employed, Poor)

score(Yes) = 0.5 × 1/5 × 1/5 × 1/5 = 0.5 × 0.2 × 0.2 × 0.2 = 0.004

score(No) = 0.5 × 3/5 × 4/5 × 4/5 = 0.5 × 0.6 × 0.8 × 0.8 = 0.192

Normalised: P(No | x) = 0.192 / 0.196 = 0.9796, P(Yes | x) = 0.004 / 0.196 = 0.0204 ⟹ Reject (No)

Application (Medium, Salaried, Good)

score(Yes) = 0.5 × 2/5 × 4/5 × 4/5 = 0.5 × 0.4 × 0.8 × 0.8 = 0.128

score(No) = 0.5 × 1/5 × 1/5 × 1/5 = 0.004

Normalised: P(Yes | x) = 0.128 / 0.132 = 0.9697, P(No | x) = 0.004 / 0.132 = 0.0303 $\Longrightarrow$ Approve (Yes)

3. Unseen value Income = "Very High" – failure of MLE and Laplace smoothing

Test application: (Very High, Salaried, Good). "Very High" never occurs in the training data, so the maximum-likelihood estimates are P(Very High | Yes) = 0/5 = 0 and P(Very High | No) = 0/5 = 0. Both unnormalised scores become 0 (log-score $-\infty$), the normalising constant is 0 and the posterior is 0/0 – undefined. A single zero count wipes out all the other evidence (Salaried and Good both point strongly to Yes), so the classifier cannot decide.

Laplace (add-α) smoothing: $P(v \mid c) = (\text{count}(v, c) + \alpha) / (\text{count}(c) + \alpha \cdot k)$, where k is the number of possible values of the feature (Income now has k = 4 values: High, Medium, Low, Very High; Employment and Credit have k = 2). **Chosen α = 1** (classic Laplace) because the sample is tiny (5 records per class) and a prior of one pseudo-observation per value is the standard neutral choice.

P(Very High | Yes) = (0 + 1) / (5 + 4) = 1/9 P(Very High | No) = 1/9

P(Salaried | Yes) = (4 + 1) / (5 + 2) = 5/7 P(Good | Yes) = 5/7

P(Salaried | No) = (1 + 1) / (5 + 2) = 2/7 P(Good | No) = 2/7

score(Yes) = 0.5 × 1/9 × 5/7 × 5/7 = 0.02834 score(No) = 0.5 × 1/9 × 2/7 × 2/7 = 0.00454

P(Yes | x) = 0.02834 / 0.03288 = 0.862 P(No | x) = 0.138 $\Longrightarrow$ Approve

Effect of α (same application):

α	**score(Yes)**	**score(No)**	**P(Yes | x)**	**Comment**
0 (MLE)	0	0	undefined	fails
0.1	0.005756	0.000414	0.933	close to MLE, over-confident
0.5	0.020089	0.002232	0.900	–
1.0	0.028345	0.004535	0.862	chosen; most conservative

Larger α shrinks the estimates towards the uniform distribution (less confident, lower variance); smaller α trusts the counts more (closer to MLE). The prediction (Approve) is the same for every α > 0, but the confidence changes from 0.93 to 0.86.

4. Implementation from scratch (log-probabilities) and library verification

The class below stores counts, adds α to numerator and α·k to the denominator, and sums log-probabilities (an unseen value with α = 0 gives log 0 = −∞). Posteriors are obtained by subtracting the maximum log-score before exponentiating (log-sum-exp).

Verification: scikit-learn's CategoricalNB(alpha=1.0, min_categories=[4,2,2]) was fitted on the same ordinally-encoded data; the largest absolute difference in P(Yes | x) over the 10 training rows, the two manual applications and the "Very High" application is 2.2×10^{-16} (floating-point rounding), and the manual hand calculations above match the code output (0.0204 / 0.9697 / 0.862).

```
class NB:
    def __init__(self, alpha=0.0): self.alpha = alpha
    def fit(self, X, y, vocab=None):
        self.cls = sorted(set(y)); self.n = len(y); d = len(X[0])
        self.vocab = vocab or [sorted({r[j] for r in X}) for j in range(d)]
        self.cnt = {c: Counter(y)[c] for c in self.cls}
        self.cond = {(c, j): Counter(r[j] for r, yy in zip(X, y) if yy == c) for c in self.cls for j in range(d)}
        return self
    def logp(self, x):                  # unnormalised log-posterior for each class
        out = {}
        for c in self.cls:
            s = math.log(self.cnt[c]/self.n)
            for j, v in enumerate(x):
                num = self.cond[(c, j)][v] + self.alpha; den = self.cnt[c] + self.alpha*len(self.vocab[j])
                s += math.log(num/den) if num > 0 else -math.inf
            out[c] = s
        return out
    def proba(self, x):
        l = self.logp(x); m = max(l.values())
        if m == -math.inf: return None
        e = {c: math.exp(v-m) for c, v in l.items()}; z = sum(e.values()); return {c: v/z for c, v in e.items()}
```

5. Redundant-feature experiment

Data-generation procedure. Class Y ~ Bernoulli(0.5). Three binary features, conditionally independent given Y: P(X1 = 1 | Y = 1, 0) = 0.75, 0.25; P(X2 = 1 | Y = 1, 0) = 0.65, 0.35; P(X3 = 1 | Y = 1, 0) = 0.60, 0.40. The redundant version adds X4 = X1 (exact duplicate). For each of 20 seeds (NumPy default_rng(100 + s), s = 0…19) a training set of 1,000 records and an independent test set of 5,000 records are generated; Naïve Bayes (α = 1) is fitted with and without X4. Log-loss is the mean negative log-probability of the true class; ECE is the 10-bin expected calibration error.

Model	Accuracy	Log-loss	ECE	Records
NB, 3 features (no redundancy)	0.7468 ± 0.0057	0.5156 ± 0.0057	0.0184	1,000 / 5,000
NB, 4 features (X4 = X1)	0.7468 ± 0.0055	0.6044 ± 0.0255	0.1438	1,000 / 5,000

(Values are means ± standard deviation over the 20 seeds. For a single seed, seed = 100: accuracy 0.7392 → 0.7420, log-loss 0.5187 → 0.5949, ECE 0.0133 → 0.1376.)

Validation Inquiry – Answer

Does a violated independence assumption always reduce accuracy? No. Duplicating X1 leaves accuracy unchanged on average (0.7468 vs 0.7468) because the duplicate only multiplies the weight of the X1 evidence in the log-odds by two; the predicted class can change only for records close to the decision boundary, and here the net effect on accuracy is nil. Naïve Bayes needs only the correct ranking of the classes, which is robust to dependence.

Accuracy versus reliability of probabilities. The predicted probabilities, however, become unreliable: the log-loss rises from 0.516 to 0.604 (+17 %) and ECE grows from 0.018 to 0.144 (eight-fold). The duplicated evidence is counted twice, so the logit contribution of X1 doubles from ±1.10 to ±2.20 and the posteriors are pushed towards 0 and 1 – the model becomes over-confident. A user who thresholds at 0.5 would see no damage, but a user who relies on P(Approve) for pricing, risk or expected cost would be misled.

Why log-space is necessary. The likelihood is a product of many small probabilities; with d features of probability about 0.01, the product 10^(−2d) underflows IEEE double precision (smallest positive ≈ 10^−308) for d ≥ 155 and becomes exactly 0 for every class, so the argmax is lost. Summing

logarithms is numerically stable, turns the product into a sum that is cheap to compute, and class posteriors are then recovered with the log-sum-exp trick (subtract the maximum log-score before exponentiating), as in the implementation above.

Implementation of the experiment (Python)

```
def gen(n, rng, dup):
    y = rng.integers(0, 2, n); p = np.where(y[:, None] == 1, [0.75, 0.65, 0.60], [0.25, 0.35, 0.40])
    X = (rng.random((n, 3)) < p).astype(int)
    if dup: X = np.hstack([X, X[:, [0]]])       # exact duplicate of feature 1
    return X, y

def fit_bin(X, y, a=1.0):
    pri = np.array([(y == c).mean() for c in (0, 1)]); th = np.array([[(X[y == c, j].sum()+a)/((y == c).sum()+2*a) for j in range(X.shape[1])] for c in (0, 1)])
    return pri, th

def pred_bin(m, X):
    pri, th = m; ll = np.log(pri)[None, :] + (X[:, None, :]*np.log(th)[None] + (1-X[:, None, :])*np.log(1-th)[None]).sum(2)
    ll -= ll.max(1, keepdims=True); p = np.exp(ll); return p/p.sum(1, keepdims=True)

def logloss(p, y): return -np.mean(np.log(np.clip(p[np.arange(len(y)), y], 1e-12, 1)))

def ece(p, y, B=10):
    conf = p[:, 1]; e = 0
    for b in range(B):
        mk = (conf >= b/B) & (conf < (b+1)/B + (b == B-1))
        if mk.any(): e += mk.mean()*abs(conf[mk].mean() - y[mk].mean())
    return e
```

Q4 – MODEL AND QUERY A BAYESIAN BELIEF NETWORK

1. Network, factored joint distribution and parameter count

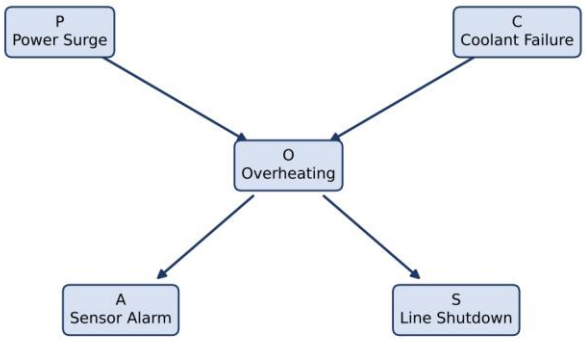


Figure Q4.1 – Network structure: P → O ← C (collider), O → A, O → S

$$P(P, C, O, A, S) = P(P) \cdot P(C) \cdot P(O \mid P, C) \cdot P(A \mid O) \cdot P(S \mid O)$$

Factor	Parents	CPT rows	Free parameters
P(P)	none	1	1
P(C)	none	1	1
P(O | P, C)	P, C	4	4
P(A | O)	O	2	2
P(S | O)	O	2	2
Total			10

The full joint distribution over five binary variables has $2^5 - 1 =$ **31** free parameters; the network needs only **10** (a 68 % reduction) because it encodes the conditional independences of the causal structure.

2. d-separation claims

Statement	Holds?	Justification (path analysis)
$A \perp S \mid O$	Yes	The only path A ← O → S is a fork (common cause) at O; observing O blocks it. Numerically P(A=T, S=T | O=T) = 0.63 = 0.9 × 0.7.
$P \perp C$ (no evidence)	Yes	The only path P → O ← C has a collider at O, and neither O nor its descendants (A, S) are observed, so the path is blocked. Numerically P(P=T, C=T) = 0.005 = 0.10 × 0.05.

Statement	Holds?	Justification (path analysis)
$P \perp C \mid O$	No	Conditioning on the collider O opens the path. Numerically $P(P=T \mid O=T, C=T) = 0.117 \neq P(P=T \mid O=T) = 0.538$.
$P \perp C \mid A$	No	A is a descendant of the collider O; observing a descendant of a collider also opens the path (more weakly). Numerically $P(C=T \mid A=T, P=T) = 0.0746 \neq P(C=T \mid A=T) = 0.2516$.

3. Manual computation of P(O = T) and P(C = T | A = T)

P(O = T) = $\Sigma_{P,C}$ P(P) P(C) P(O=T | P, C):

P, C	P(P)	P(C)	P(O=T | P,C)	Product
T, T	0.10	0.05	0.95	0.00475
T, F	0.10	0.95	0.60	0.05700
F, T	0.90	0.05	0.80	0.03600
F, F	0.90	0.95	0.02	0.01710
Sum				**0.11485**

P(C = T | A = T). First P(A = T | P, C) = P(O=T | P,C)·0.90 + P(O=F | P,C)·0.05 = 0.05 + 0.85·P(O=T | P,C), giving 0.8575 for (T,T), 0.56 for (T,F), 0.73 for (F,T) and 0.067 for (F,F).

P(A = T) = 0.9 × 0.11485 + 0.05 × 0.88515 = 0.103365 + 0.044258 = 0.147623

P(C = T, A = T) = P(C=T) Σ_P P(P) P(A=T | P, C=T) = 0.05 × (0.10 × 0.8575 + 0.90 × 0.73) = 0.05 × 0.74275 = 0.037138

P(C = T | A = T) = 0.037138 / 0.147623 = 0.2516

4. Explaining away: P(C = T | A = T, P = T)

C = T: P(P=T) P(C=T) P(A=T | P=T, C=T) = 0.10 × 0.05 × 0.8575 = 0.0042875

C = F: P(P=T) P(C=F) P(A=T | P=T, C=F) = 0.10 × 0.95 × 0.56 = 0.0532

P(C = T | A = T, P = T) = 0.0042875 / (0.0042875 + 0.0532) = 0.0746

Explanation. The alarm is evidence of overheating, and overheating makes both causes more likely: P(C = T) rises from the prior 0.05 to 0.2516 after the alarm. When we then learn that a power surge occurred – an alternative cause that already accounts for the overheating – the coolant failure is no longer needed to explain the alarm, so its probability falls to 0.0746 (still above the prior because a surge is not a certain cause). Two marginally independent causes of a common effect become

negatively dependent once the effect (or a descendant) is observed; this is the collider behaviour found in the d-separation analysis.

5. Inference by enumeration (Python) and number of joint terms

The function sums the full joint P(P,C,O,A,S) over all assignments of the hidden variables, once for each value of the query variable, and normalises. All coded answers equal the manual ones.

Query	Manual	Code	Joint terms summed
P(O = T)	0.11485	0.11485	16 (hidden P, C, A, S = 2^4)
P(C = T | A = T)	0.2516	0.25157	16 (8 for C = T + 8 for C = F; hidden P, O, S)
P(C = T | A = T, P = T)	0.0746	0.07458	8 (4 + 4; hidden O, S)

```
def pP(p): return 0.10 if p else 0.90

def pO(o,p,c): return OT[(p,c)] if o else 1-OT[(p,c)]

def joint(P,C,O,A,S): return pP(P)*pC(C)*pO(O,P,C)*pA(A,O)*pS(S,O)

def enum(query, evidence):
    """P(query var | evidence) by summing the full joint over all hidden variables."""
    global terms; terms = 0; qv = list(query)[0]; res = {}
    for qval in (1, 0):
        tot = 0.0; fixed = dict(evidence); fixed[qv] = qval
        hidden = [v for v in VARS if v not in fixed]
        for vals in itertools.product((0,1), repeat=len(hidden)):
            a = dict(fixed); a.update(zip(hidden, vals)); tot += joint(*(a[v] for v in VARS)); terms += 1
        res[qval] = tot
    z = sum(res.values()); return res[1]/z, terms, res
```

6. Simulation and CPT estimation (seed = 7, 1,000 records)

Records are generated by ancestral sampling (P, C, then O, A, S). The sample contained only 5 records with P = T and C = T. CPTs were estimated by maximum likelihood (relative frequencies) from the first 50, 200 and 1,000 records. Error = |estimate − true value|.

Row (P,C)	True P(O=T | ·)	n = 50	n = 200	n = 1000
T, T	0.95	undefined (0 records)	1.000 (1 rec.), err 0.050	0.800 (5 rec.), err 0.150
T, F	0.60	0.667 (6 rec.), err 0.067	0.500 (20 rec.), err 0.100	0.592 (98 rec.), err 0.008
F, T	0.80	1.000 (3 rec.), err 0.200	0.833 (6 rec.), err 0.033	0.750 (36 rec.), err 0.050
F, F	0.02	0.024 (41 rec.), err 0.004	0.006 (173 rec.), err 0.014	0.021 (861 rec.), err 0.001
Mean error (defined rows)		0.090	0.049	0.052

Estimates of the remaining CPTs from the same samples:

Sample	P(P)	P(C)	P(A|O=T)	P(A|O=F)	P(S|O=T)	P(S|O=F)
True	0.100	0.050	0.900	0.050	0.700	0.010
n = 50	0.120	0.060	1.000	0.024	0.750	0.000
n = 200	0.105	0.035	1.000	0.049	0.882	0.005
n = 1000	0.103	0.041	0.869	0.045	0.701	0.010

Reliability of the single run. Because one sample is noisy, the experiment was repeated 300 times per sample size (different seeds, 1000 + s). Mean absolute error of P(O = T | P, C), by row:

Sample size	(T,T)	(T,F)	(F,T)	(F,F)	Row (T,T) undefined in
n = 50	0.081	0.201	0.230	0.016	76.7 % of runs
n = 200	0.094	0.090	0.112	0.009	33.7 % of runs
n = 1000	0.080	0.043	0.049	0.004	0.7 % of runs

Interpretation. The error of a maximum-likelihood CPT row shrinks roughly as $1/\sqrt{}$(number of records in that row). Common parent configurations (F,F: 86 % of the data) are estimated accurately even from 50 records, whereas rare configurations (T,T has probability 0.005) have an expected 0.25

records in a sample of 50 and are frequently undefined, and even with 1,000 records they remain poorly determined (error ≈ 0.08–0.15). Hence ML estimates for rare parent configurations need smoothing (Dirichlet / Laplace priors) or more data.

Design Inquiry

Independence assumption relaxed. Naïve Bayes assumes that all features are conditionally independent given the class (a star-shaped network with only the class as a parent). The belief network relaxes this to a local Markov assumption: each variable is independent of its non-descendants given its parents. It allows dependencies among variables (here P and C jointly cause O, and A and S are dependent unless O is known) while keeping them sparse. Naïve Bayes is the special case of a belief network with one parent (the class) for every feature. The price is more parameters (each node with k parents needs 2^k CPT rows) and the need to specify or learn the structure.

Cost of exact inference. Inference by enumeration sums over 2^(number of hidden variables) joint terms: 16 terms already for five variables with one query, and doubling with every additional hidden binary variable. Variable elimination reduces this to a cost exponential in the treewidth of the graph, but exact inference in general belief networks is NP-hard. For large or densely connected networks one therefore uses approximate inference (sampling, loopy belief propagation, variational methods).

Q5 – DEVELOP AND ANALYSE THE EM ALGORITHM FOR A MIXTURE MODEL

1. Mixture model, E-step, M-step and derivation of the μ_k update

$$p(x \mid \theta) = \Sigma_{k=1}^{2} \pi_k N(x \mid \mu_k, \sigma_k^2), \quad \pi 1 + \pi 2 = 1, \quad N(x \mid \mu, \sigma^2) = (2\pi\sigma^2)^{(-1/2)} \exp[-(x - \mu)^2 / (2\sigma^2)]$$

Parameters: $\theta = \{\pi_k, \mu_k, \sigma_k^2\}$, $k = 1, 2$. **Hidden variable:** $z_i \in \{1, 2\}$, the unrecorded machine that produced reading x_i, with $P(z_i = k) = \pi_k$ and $x_i \mid z_i = k \sim N(\mu_k, \sigma_k^2)$.

E-step (responsibility = posterior probability of machine k for reading i under the current parameters):

$$r_{ik} = P(z_i = k \mid x_i, \theta) = \pi_k N(x_i \mid \mu_k, \sigma_k^2) / \Sigma_j \pi_j N(x_i \mid \mu_j, \sigma_j^2)$$

M-step (with $N_k = \Sigma_i r_{ik}$ and N the number of readings):

$$\pi_k = N_k / N \qquad \mu_k = (1/N_k) \Sigma_i r_{ik} x_i \qquad \sigma_k^2 = (1/N_k) \Sigma_i r_{ik} (x_i - \mu_k)^2$$

Derivation of μ_k. The expected complete-data log-likelihood is

$$Q(\theta \mid \theta_{old}) = \Sigma_i \Sigma_k r_{ik} [\ln \pi_k - \tfrac{1}{2} \ln(2\pi\sigma_k^2) - (x_i - \mu_k)^2 / (2\sigma_k^2)]$$

Only the last term depends on μ_k. Setting the derivative to zero:

$$\partial Q/\partial \mu_k = \Sigma_i r_{ik} (x_i - \mu_k) / \sigma_k^2 = 0 \implies \Sigma_i r_{ik} x_i = \mu_k \Sigma_i r_{ik} \implies \mu_k = \Sigma_i r_{ik} x_i / \Sigma_i r_{ik}$$

i.e. μ_k is the responsibility-weighted mean (the second derivative is $-N_k/\sigma_k^2 < 0$, so this is a maximum). In the same way σ_k^2 follows from $\partial Q/\partial \sigma_k^2 = 0$, and π_k from maximising $\Sigma_{ik} r_{ik} \ln \pi_k$ subject to $\Sigma \pi_k = 1$ with a Lagrange multiplier, giving $\pi_k = N_k/N$.

2. One manual EM iteration on the seven readings

Start: $\pi 1 = \pi 2 = 0.5$, $\mu 1 = 2$, $\mu 2 = 8$, $\sigma 1^2 = \sigma 2^2 = 1$. Weighted densities $w_{ik} = \pi_k N(x_i \mid \mu_k, 1)$; the mixture density is $p(x_i) = w_{i1} + w_{i2}$ and $r_{ik} = w_{ik} / p(x_i)$.

i	x_i	w_i1	w_i2	r_i1	r_i2	ln p(x_i)
1	1.8	0.195521	≈ 0	1.0000	0.0000	−1.6321
2	2.5	0.176033	≈ 0	1.0000	0.0000	−1.7371
3	3.1	0.108926	0.000001	1.0000	0.0000	−2.2171
4	5.0	0.002216	0.002216	0.5000	0.5000	−5.4189
5	7.6	≈ 0	0.184135	0.0000	1.0000	−1.6921
6	8.4	≈ 0	0.184135	0.0000	1.0000	−1.6921
7	9.0	≈ 0	0.120985	0.0000	1.0000	−2.1121

Log-likelihood before: $\Sigma \ln p(x_i) = -1.6321 - 1.7371 - 2.2171 - 5.4189 - 1.6921 - 1.6921 - 2.1121 = \mathbf{-16.5014}$.

M-step. $N1 = 1 + 1 + 1 + 0.5 + 0 + 0 + 0 = 3.5$, $N2 = 3.5$.

$\pi 1 = \pi 2 = 3.5 / 7 = 0.5$

$\mu 1 = (1.8 + 2.5 + 3.1 + 0.5 \times 5.0) / 3.5 = 9.9 / 3.5 = 2.8286$ $\mu 2 = (0.5 \times 5.0 + 7.6 + 8.4 + 9.0) / 3.5 = 27.5 / 3.5 = 7.8571$

$\sigma 1^2 = [(1.8-2.8286)^2 + (2.5-2.8286)^2 + (3.1-2.8286)^2 + 0.5(5.0-2.8286)^2] / 3.5 = 3.5970 / 3.5 = 1.0278$

$\sigma 2^2 = [0.5(5.0-7.8571)^2 + (7.6-7.8571)^2 + (8.4-7.8571)^2 + (9.0-7.8571)^2] / 3.5 = 5.7485 / 3.5 = 1.6425$

Log-likelihood after (recomputed with the new parameters): **−14.9842**, an increase of 1.517, as guaranteed. The reading 5.0 sits exactly between the two means and has responsibility 0.5 for each machine.

3. Implementation:

Dataset (reproducible): NumPy default_rng(2024); 500 readings from the known mixture $\pi = (0.4, 0.6)$, $\mu = (2, 7)$, $\sigma = (1.0, 1.5)$. **Initialisation:** $\pi = (0.5, 0.5)$, $\mu = (\min + 1, \max - 1)$, $\sigma^2 = (1, 1)$. **Convergence rule:** stop when $|LL_t - LL_{t-1}| < 10^{-8}$ or after 1,000 iterations; a variance floor of 10^{-6} prevents component collapse. **Result:** converged after 58 iterations to $\pi = (0.3614, 0.6386)$, $\mu = (2.038, 6.989)$, $\sigma = (1.007, 1.648)$ with LL = −1154.2685. The log-likelihood increased at every iteration (smallest increment 8.1×10^{-9}, none negative), as the plot shows.

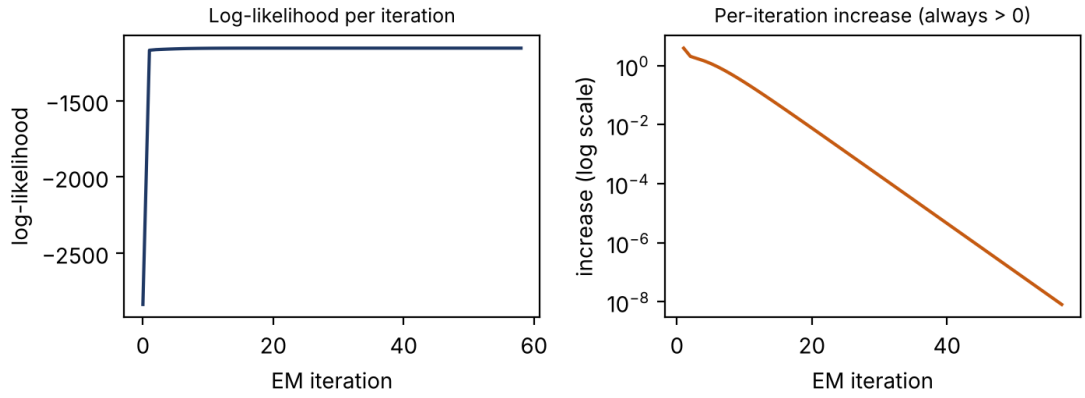


Figure Q5.1 – Log-likelihood per EM iteration (left) and per-iteration increase on a log scale (right)

```
def npdf(x, mu, var): return np.exp(-(x-mu)**2/(2*var))/np.sqrt(2*np.pi*var)


def loglik(x, pi, mu, var): return np.log((pi[None,:]*npdf(x[:,None], mu[None,:], var[None,:])).sum(1)).sum()
```

```
def estep(x, pi, mu, var):
    w = pi[None,:]*npdf(x[:,None], mu[None,:], var[None,:]); return w/w.sum(1, keepdims=True)

def mstep(x, r, floor=1e-6):
    Nk = r.sum(0); pi = Nk/len(x); mu = (r*x[:,None]).sum(0)/Nk; var = (r*(x[:,None]-mu[None,:])**2).sum(0)/Nk; return pi, mu, np.maximum(var, floor)

def em(x, pi, mu, var, tol=1e-8, maxit=1000):
    pi, mu, var = map(np.array, (pi, mu, var)); ll = [loglik(x, pi, mu, var)]
    for it in range(1, maxit+1):
        r = estep(x, pi, mu, var); pi, mu, var = mstep(x, r); ll.append(loglik(x, pi, mu, var))
        if abs(ll[-1]-ll[-2]) < tol: break
    return pi, mu, var, np.array(ll), it
```

4. Ten random initialisations (K = 2)

Means drawn uniformly from the data range (not sorted, so labels may swap), variances uniform on [0.5, 4], mixing weights from a Dirichlet(2, 2); runs 8–10 were deliberately poor starts (nearly equal means).

Run	Initial μ1, μ2	Iterations	Final LL	π1, π2	μ1, μ2	σ1, σ2
1	0.94, 5.34	58	−1154.269	0.361, 0.639	2.038, 6.989	1.007, 1.648
2	10.67, 6.80	88	−1154.269	0.639, 0.361	6.989, 2.038	1.648, 1.007
3	7.37, 5.49	78	−1154.269	0.639, 0.361	6.989, 2.038	1.648, 1.007
4	3.60, 6.44	58	−1154.269	0.361, 0.639	2.038, 6.989	1.007, 1.648
5	0.40, 10.05	63	−1154.269	0.361, 0.639	2.038, 6.989	1.007, 1.648
6	−0.20, 1.79	60	−1154.269	0.361, 0.639	2.038, 6.989	1.007, 1.648
7	1.31, 11.24	62	−1154.269	0.361, 0.639	2.038, 6.989	1.007, 1.648
8	3.08, 3.18	57	−1154.269	0.361, 0.639	2.038, 6.989	1.007, 1.648
9	9.00, 9.50	89	−1154.269	0.361, 0.639	2.038, 6.989	1.007, 1.648
10	5.00, 5.20	80	−1154.269	0.361, 0.639	2.038, 6.989	1.007, 1.648

Observations. (i) All ten runs reach the same log-likelihood (−1154.269), so for K = 2 on this dataset there is **no inferior local optimum**; the poor starts of runs 9 and 10 only needed more iterations (89 and 80, against 57–63 for most other runs), and runs 2 and 3 also took longer (88 and 78). (ii) **Label switching** occurs in runs 2 and 3: they start with $\mu 1 > \mu 2$ and end with the same solution with the component labels exchanged ($\pi = (0.639, 0.361)$, $\mu = (6.989, 2.038)$); it is the same mixture, so the likelihood is identical. (iii) For K = 3 the surface is much flatter: ten random starts gave final log-likelihoods between −1153.35 and −1153.68 with different component layouts (e.g. means (1.61, 2.28, 7.08) versus (1.95, 6.21, 7.67)) and none met the 10^{-8} rule within 1,000 iterations – distinct near-optima and slow drift along a ridge, which is the typical local-optimum behaviour of EM.

5. Choosing the number of components (best of 10 starts for each K)

K	Log-likelihood	Free parameters 3K − 1	BIC = −2LL + p ln 500	AIC = −2LL + 2p	Decision
1	−1221.615	2	2455.66	2447.23	
2	−1154.269	5	**2339.61**	**2318.54**	**selected**
3	−1153.353	8	2356.42	2322.71	

Moving from K = 1 to K = 2 raises the log-likelihood by 67.3, far more than the penalty. Moving from K = 2 to K = 3 gains only 0.92 log-likelihood units for three additional parameters, while the BIC penalty rises by 3 ln 500 = 18.6; BIC (and AIC) are lowest at K = 2, so **the data support two components**, matching the generating process.

6. Soft (EM) versus hard (k-means style) assignment, K = 2

Quantity	EM (soft)	Hard assignment	True value
$\pi 1, \pi 2$	0.361, 0.639	0.432, 0.568	0.4, 0.6
$\mu 1, \mu 2$	2.038, 6.989	2.378, 7.345	2, 7
$\sigma 1, \sigma 2$	1.007, 1.648	1.228, 1.361	1.0, 1.5
Log-likelihood of the 500 points	−1154.27	−1161.60	–
Accuracy against true generating machine	97.2 %	94.0 %	–

The two methods agree on 93.6 % of the points; the difference lies in the overlap region: 53 readings have a maximum responsibility below 0.9. Hard assignment forces each of them wholly into one cluster, so the cluster statistics are estimated from truncated data (e.g. $\sigma 1 = 1.23$ against the true 1.0 and $\mu 1 = 2.38$ against 2), the weights are distorted and the likelihood is lower by 7.3. EM keeps fractional membership for ambiguous points, recovers the weights, means and $\sigma 1$ more closely ($\sigma 2$ is about equally accurate in both) and gives assignment probabilities rather than only labels. Hard assignment is a special case (responsibilities rounded to 0/1) and is cheaper.

Why the likelihood never decreases. For any distribution $q(z)$ over the hidden variables, $\ln p(X \mid \theta) = L(q, \theta) + KL(q \parallel p(Z \mid X, \theta))$, where $L(q, \theta) = \Sigma\, q(z) \ln[p(X, z \mid \theta)/q(z)]$ is a lower bound (Jensen's inequality, $KL \geq 0$). The E-step sets $q = p(Z \mid X, \theta_old)$, making $KL = 0$, so $L(q, \theta_old) = \ln p(X \mid \theta_old)$: the bound touches the likelihood at θ_old. The M-step maximises $L(q, \theta)$ over θ, so $L(q, \theta_new) \geq L(q, \theta_old)$. Finally $\ln p(X \mid \theta_new) \geq L(q, \theta_new) \geq L(q, \theta_old) = \ln p(X \mid \theta_old)$. Each iteration therefore cannot decrease the likelihood.

Why this does not guarantee a global maximum. EM climbs the likelihood until it reaches a stationary point, where the bound can no longer be improved; the mixture likelihood is not concave, so there are many such points (permutation copies, local maxima, saddle points, and in the Gaussian case singular solutions where a component collapses onto one point and the likelihood tends to infinity). The point reached depends on the initialisation, as seen in the $K = 3$ runs, which is why we use multiple restarts, a variance floor and model-selection criteria.

Learning the CPTs of the Q4 network with unobserved variables. Suppose the overheating variable O was never recorded. **E-step:** for every record compute the posterior $P(O \mid P, C, A, S)$ by inference in the network (enumeration from Q4). **M-step:** re-estimate each CPT from expected counts, e.g. $P(O = T \mid P = p, C = c) = \Sigma_i\, P(O = T \mid \text{record } i) \cdot 1[P_i = p, C_i = c] / \Sigma_i\, 1[P_i = p, C_i = c]$, $P(A = T \mid O = T) = \Sigma_i\, P(O = T \mid \text{record } i)\, A_i / \Sigma_i\, P(O = T \mid \text{record } i)$, and similarly for $S \mid O$. Iterate until the log-likelihood stops improving. Exactly the same lower-bound argument gives monotonic improvement, and the same cautions about local optima and label symmetry (here the meaning of O is fixed only by the CPTs' initialisation) apply.

Q6 – INVESTIGATE SAMPLE COMPLEXITY, HYPOTHESIS SPACE CAPACITY AND MISTAKE BOUNDS

Part A – PAC learning and finite hypothesis spaces

PAC framework. A concept class C is PAC-learnable by a learner using hypothesis space H if, for every distribution D over the instances, every target c ∈ C and all ε, δ ∈ (0, 1), the learner outputs, with probability at least 1 − δ over the random training sample, a hypothesis h whose true error error_D(h) = P_{x~D}[h(x) ≠ c(x)] is at most ε, using a number of examples (and time) polynomial in 1/ε, 1/δ, n and the size of c. "Probably (1 − δ) approximately (ε) correct." For a consistent learner in a finite H, the probability that some hypothesis with error > ε is consistent with m examples is at most |H|(1 − ε)^m ≤ |H|e^(−εm) ≤ δ, which gives m ≥ (1/ε)(ln|H| + ln(1/δ)).

Sample sizes for n = 10 (|H| = 3^{10} + 1 = 59,050; ln|H| = 10.9861):

ε	δ	(1/ε)(ln|H| + ln(1/δ))	m (rounded up)	ln(1/δ)
0.10	0.05	139.82	140	3.00
0.10	0.01	155.91	156	4.61
0.05	0.05	279.64	280	3.00
0.05	0.01	311.83	312	4.61

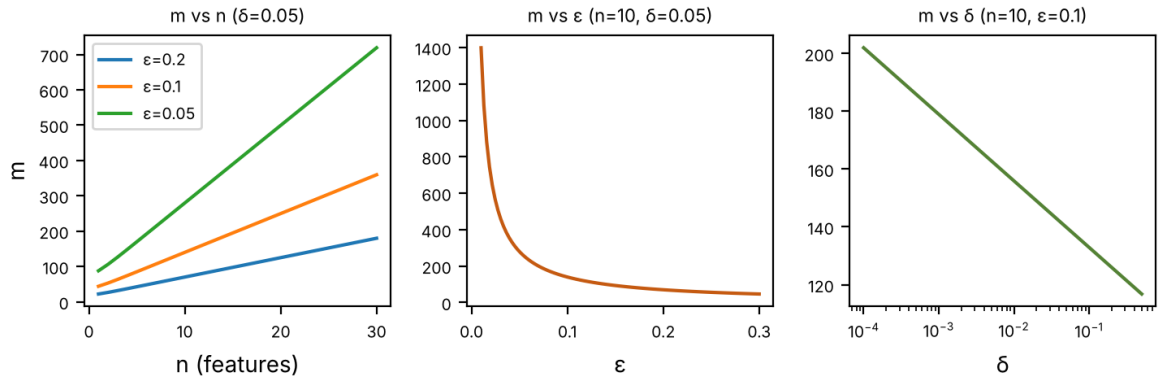


Figure Q6.1 – PAC bound versus n, ε and δ

Strongest influence. The bound grows linearly in n (≈ n ln 3 inside the bracket), as 1/ε, and only logarithmically in 1/δ. In the table above, halving ε from 0.10 to 0.05 doubles the sample size (× 2.00); moving δ from 0.05 to 0.01 (a five-fold tighter confidence) increases it by only 11.5 % (× 1.115); doubling n from 10 to 20 raises it by 79 % (× 1.79). **ε is the dominant parameter**, n second, and δ the weakest.

Find-S simulation. Setting: n = 10 Boolean features drawn uniformly from $\{0,1\}^{10}$, target = the single-literal conjunction x1 (the target with the largest positive region, P(target) = 0.5, so the error can be as large as 0.5). Find-S starts with the most specific hypothesis (all 20 literals) and deletes literals falsified by each positive example. The true error is computed exactly: since the learned conjunction h always contains the target's literals, error(h) = P(target) − P(h) = 2^−|c| − 2^−|h|. Seed 123, 500 trials per sample size.

m	**Mean true error**	**Fraction of trials with error ≤ 0.1**	**Comment**
4	0.4226	0.054	
8	0.2367	0.346	
12	0.0898	0.714	
16	0.0386	0.866	
20	0.0146	0.944	≈ 95 % reached at m ≈ 21
25	0.0025	0.990	
30	0.0005	0.998	
40	0.0000	1.000	
60	0.0000	1.000	
100	0.0000	1.000	
140	0.0000	1.000	= PAC bound (ε=0.1, δ=0.05)

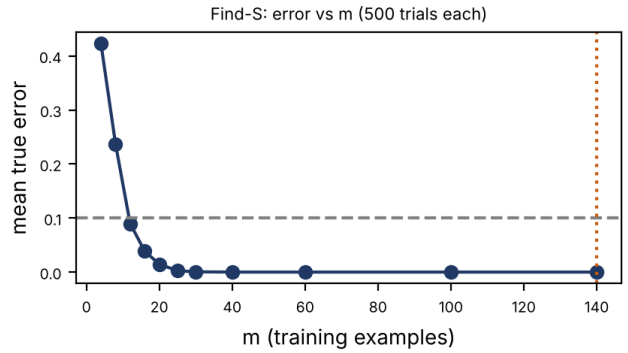


Figure Q6.2 – Find-S: mean true error against m (dashed: ε = 0.1; dotted: bound m = 140)

Check at the bound. At m = 140 (ε = 0.1, δ = 0.05) the fraction of trials with error ≤ ε is 1.000 ≥ 0.95; at m = 156 (ε = 0.1, δ = 0.01) it is 1.000 ≥ 0.99; at m = 280 (ε = 0.05, δ = 0.05) it is 1.000 ≥ 0.95. The

guarantee holds in every case. (For this target the error can only be 0 or ≥ 0.25, so the fractions for ε = 0.1 and 0.05 coincide.)

How conservative is the bound? The empirical sample size needed for 95 % of runs to have error ≤ 0.1 is about 21, versus 140 from the bound, i.e. the bound is about **6.7 times too large** here. It is a worst-case guarantee over all distributions, all targets and all consistent learners; it uses a union bound over the 59,050 hypotheses; and the uniform distribution with independent features is far friendlier to Find-S than a worst-case distribution, because every irrelevant literal is eliminated with probability ½ by each positive example.

Part B – Infinite hypothesis spaces

Definitions. A set S of instances is *shattered* by H if for every one of the 2^|S| labellings of S there is a hypothesis in H that realises it. The *VC dimension* VC(H) is the size of the largest finite set shattered by H (infinite if arbitrarily large sets can be shattered). To show VC(H) = d one must exhibit one shattered set of d points and prove that no set of d + 1 points can be shattered. All claims below were also verified by brute force in the code.

Class	**Shattered set of size VC**	**Why no larger set is shattered**
Intervals [a, b] on the real line	VC = 2. Points {1, 2}: ++ → [1, 2]; +− → [1, 1.5]; −+ → [1.5, 2]; −− → [3, 4] (empty on the points).	For any x1 < x2 < x3 the labelling (+, −, +) is impossible: an interval containing x1 and x3 contains x2.
Linear separators (lines) in the plane	VC = 3. Non-collinear points (0,0), (1,0), (0,1): all 8 labellings are separable by a line (verified with a linear programme).	By Radon's theorem any 4 points in the plane can be split into two sets with intersecting convex hulls, which no line separates (e.g. the XOR labelling of a square, or the three outer points of a triangle labelled + with the inner point labelled −).
Axis-aligned rectangles in the plane	VC = 4. The diamond (0,1), (1,0), (2,1), (1,2): for every subset the bounding rectangle of the subset contains none of the other points, so every labelling is realised.	For any 5 points let the leftmost, rightmost, topmost and bottommost points (at most 4 distinct points) be labelled + and the remaining point −: every rectangle containing the extreme points contains their bounding box, which contains the fifth point.

VC-based sample bound for $\varepsilon = 0.1$, $\delta = 0.05$: $4 \log_2(2/\delta) = 4 \log_2 40 = 21.29$ and $\log_2(13/\varepsilon) = \log_2 130 = 7.0224$.

Class	**VC(H)**	**$(1/\varepsilon)(21.29 + 8 \cdot VC \cdot 7.0224)$**	**m (rounded up)**
Intervals	2	(21.29 + 112.36) / 0.1 = 1336.5	1,337
Linear separators in the plane	3	(21.29 + 168.54) / 0.1 = 1898.2	1,899
Axis-aligned rectangles	4	(21.29 + 224.72) / 0.1 = 2460.0	2,461

Comparison with the finite-space bound. The finite bound uses $\ln|H|$ – the logarithm of the number of hypotheses – and therefore needs H to be finite. The VC bound replaces $\ln|H|$ by the combinatorial capacity VC(H), so it applies to infinite classes (intervals, lines, rectangles all contain infinitely many hypotheses), and the sample size grows linearly in VC(H) and, as before, as $1/\varepsilon$ up to the log factor. The price is much larger constants and an extra $\log(1/\varepsilon)$ factor: for conjunctions with $n = 10$ ($VC = 10$) the VC formula would give $(21.29 + 80 \times 7.0224)/0.1 \approx 5{,}831$ examples, compared with 140 from the finite-space bound for the same ε, δ. Both are worst-case, distribution-free guarantees; the VC bound is the only one available for infinite H.

Part C – Mistake bound model

Halving algorithm. The hypothesis space is the set of 64 parity functions $h_a(x) = (a \cdot x) \bmod 2$ on $x \in \{0,1\}^6$, $a \in \{0,1\}^6$, so $|H| = 64$ and $\log_2|H| = 6$. The learner keeps a version space V of hypotheses consistent with all examples so far, predicts by the majority vote of V, and after seeing the true label discards the inconsistent hypotheses. Whenever it errs, at least half of V is eliminated, so mistakes $\leq \log_2|H|$. Experiment: 2,000 random targets, each with a random sequence of 40 examples (seed 5).

$\log_2\|H\|$ (bound)	**Mean mistakes**	**Maximum observed**	**Distribution (0, 1, 2, … 6 mistakes)**
6	2.98	6	30, 213, 450, 642, 451, 172, 42

No run exceeded 6 mistakes; with an adversarially chosen sequence (always an instance that splits the version space and a label that contradicts the majority) the learner makes exactly 6 mistakes, so the bound is attained. Typical random sequences are far friendlier (mean 2.98) because random labels often shrink V by more than half.

Online learner for conjunctions. Start with the most specific hypothesis h = x1 ∧ ¬x1 ∧ … ∧ xn ∧ ¬xn (all 2n literals; it predicts negative for every instance). On every positive example that h classifies as negative, delete every literal that the example falsifies; negative examples are ignored.

```
def online_conjunction(n, target, examples):
    lits = {(i, v) for i in range(n) for v in (0, 1)}   # most specific hypothesis: all 2n literals
    mistakes = 0
    for x in examples:
        y = all(x[i] == v for i, v in target.items())    # true label
        h = all(x[i] == v for i, v in lits)              # prediction
        if h != y:
            mistakes += 1                                # only possible on a positive example
            lits = {(i, v) for (i, v) in lits if x[i] == v}   # drop literals falsified by x
    return mistakes
```

Proof of at most n + 1 mistakes. (1) h always contains all literals of the target, so whenever h is true the target is true: there are no false positives, and mistakes happen only on positive examples. (2) The first mistake: the example contains exactly one of x_i, ¬x_i for each i, so it deletes exactly n of the 2n literals. (3) Every later mistake is a positive example that h rejects, so it falsifies at least one literal still in h, and deletes at least one. (4) After the first mistake at most n literals remain, hence at most n further mistakes. Total ≤ 1 + n.

n	Bound n + 1	Max mistakes observed	Mean mistakes	Sequences
5	6	6	2.66	2,000
10	11	8	3.49	2,000
20	21	8	4.36	2,000

(Random target conjunctions, 60 examples per sequence with half drawn to satisfy the target, seed 77.) The bound is never exceeded and is attained for n = 5.

VC(C) ≤ Opt(C) ≤ log2|C|. Opt(C) is the smallest worst-case number of mistakes achievable by any online algorithm. *Lower bound:* if d points are shattered by C, an adversary can present them one by one and always give the label opposite to the learner's prediction; shattering guarantees a concept in C consistent with all these labels, forcing d mistakes, so Opt(C) ≥ VC(C). *Upper bound:* the Halving algorithm makes at most log2|C| mistakes, so Opt(C) ≤ log2|C|.

Confirmation on a small class. C = thresholds on four points, $c_t(x) = 1$ if $x \geq t$, for $t = 1…5$ ($|C| = 5$). Computed by brute force (VC by testing all point subsets; Opt by exhaustive minimax over the version-space game): VC(C) = 1, Opt(C) = 2, $\log_2|C| = 2.32$, so $1 \leq 2 \leq 2.32$. For the full power set on two points ($|C| = 4$): VC = Opt = $\log_2|C| = 2$, showing that the bounds can be tight on both sides. The first example shows that Opt can lie strictly between the two bounds.

Selection Inquiry

Comparison. The PAC model gives a guarantee on the *generalisation error* of a hypothesis trained on a batch of m i.i.d. examples from a fixed distribution: it says how much data to collect for error $\leq \varepsilon$ with confidence $1 - \delta$. The mistake-bound model gives a guarantee on the *total number of wrong predictions* an online learner makes on any sequence, even an adversarial one, with no distributional assumption and no δ (deterministic). A mistake-bounded algorithm can be converted into a PAC learner, but the converse is not true in general.

Which is more useful to the quality team? If the team can collect a labelled batch of inspections from a stable production process and then deploy a fixed classifier, the PAC bound is more useful: it directly answers "how many labelled bearings must we inspect?" and the guarantee concerns the future defect rate. If the learner must work online – each part is classified before its true label is known, a wrong call is costly (e.g. a defective part shipped), and the process may drift or the data order is not random – the mistake bound is more useful, because it limits the total number of wrong calls over the entire run. For conjunctions the answer is also numerically instructive: $n + 1 = 11$ mistakes in the worst case versus 140 examples for the PAC guarantee.

Assumptions in my simulations. (i) Labels are noise-free and the target is realisable (the target conjunction lies in H; the Halving target is one of the 64 parities). (ii) For the PAC simulation, instances are drawn i.i.d. from the uniform distribution over $\{0,1\}^{10}$ with independent features, and the target is a single literal – a benign distribution, so the bound is conservative. (iii) The online conjunction experiments use random examples with half of them positive; the $n + 1$ bound itself holds for every sequence. (iv) Halving sequences are random (plus one adversarial-style run). With noisy labels or an unrealisable target, Find-S, Halving and the $n + 1$ bound no longer apply and agnostic bounds are needed.

Implementation excerpts (Python, Q6)

```
def m_pac(n, eps, delta): return (math.log(3**n+1) + math.log(1/delta))/eps


def findS_error(m, rng):
    lits = {(i, v) for i in range(n) for v in (0, 1)}      # most specific hypothesis: all 2n literals
    X = rng.integers(0, 2, (m, n)); pos = np.all([X[:, i] == v for i, v in target.items()], axis=0)
```

```
    for x in X[pos]: lits = {(i, v) for (i, v) in lits if x[i] == v}
    if not pos.any(): return 2.0**-len(target)            # h = empty concept (all negative)
    # a hypothesis containing both x_i and not x_i is the empty concept; after >=1 positive it is consistent, so no contradictions
    return 2.0**-len(target) - 2.0**-len(lits)          # P(target) - P(h): h's literals contain target's

def shatter_interval(pts):
    for lab in itertools.product((0, 1), repeat=len(pts)):
        P = [p for p, l in zip(pts, lab) if l]; N = [p for p, l in zip(pts, lab) if not l]
        if P and any(min(P) <= q <= max(P) for q in N): return False
    return True

def shatter_rect(pts):
    for lab in itertools.product((0, 1), repeat=len(pts)):
        P = np.array([p for p, l in zip(pts, lab) if l]); N = [p for p, l in zip(pts, lab) if not l]
        if len(P) == 0: continue
        lo, hi = P.min(0), P.max(0)
        if any(np.all(np.array(q) >= lo) and np.all(np.array(q) <= hi) for q in N): return False
    return True

def shatter_lin(pts): return all(separable([p for p, l in zip(pts, lab) if l], [p for p, l in zip(pts, lab) if not l]) for lab in itertools.product((0, 1), repeat=len(pts)))

def m_vc(vc, eps, delta): return (4*math.log2(2/delta) + 8*vc*math.log2(13/eps))/eps

def halving(target, seq):
    V = list(Hs); mistakes = 0
    for x in seq:
        v1 = sum(f(a, x) for a in V); pred = 1 if v1 > len(V)-v1 else 0; y = f(target, x)
        mistakes += pred != y; V = [a for a in V if f(a, x) == y]
    return mistakes, len(V)

def opt(C, X):
    C = tuple(sorted(C))
    @lru_cache(None)
```

```
  def M(V):
    if len(V) <= 1: return 0
    best = 0
    for x in X:
      V1 = tuple(c for c in V if c[x] == 1); V0 = tuple(c for c in V if c[x] == 0)
      if not V1 or not V0: continue
      best = max(best, min(max(1+M(V0), M(V1)), max(M(V0), 1+M(V1))))      # learner picks prediction; adversary picks label
    return best
  return M(C)

def vcdim(C, X):
  best = 0
  for k in range(1, len(X)+1):
    for S in itertools.combinations(X, k):
      if len({tuple(c[x] for x in S) for c in C}) == 2**k: best = k
  return best
```

DAY-4 DESIGN CHALLENGE

Q7 – DIAGNOSE, REDESIGN AND VALIDATE A BAYESIAN OR COMPUTATIONAL LEARNING SOLUTION

A. DIAGNOSE THE LEARNING OR COMPUTATIONAL BOTTLENECK

Limitation. In the Q3 experiment, duplicating feature X1 left the accuracy unchanged (0.7468 → 0.7468) but degraded the probability estimates: log-loss rose from 0.5156 to 0.6044 (+17.2 %) and the expected calibration error (ECE) from 0.0184 to 0.1438 (7.8 ×). Naïve Bayes became **over-confident**.

Measure (20 seeds, 1,000 train / 5,000 test)	**3 features**	**4 features (X4 = X1)**	**Change**
Accuracy	0.7468	0.7468	0.0000
Log-loss	0.5156	0.6044	+0.0888 (+17.2 %)
ECE	0.0184	0.1438	× 7.8

Cause: the model assumption, not the data, the prior, initialisation or inference cost. (i) The decision rule multiplies P(x1 | c) and P(x4 | c) as if they were independent evidence, but X4 = X1, so the likelihood ratio of X1 is counted twice and its contribution to the log-odds doubles from ±ln 3 = ±1.10 to ±2.20. (ii) The sampling noise is small (log-loss standard deviation 0.006 on the clean data) and n = 1,000 is large for four binary features, so limited data cannot explain a 0.09 gap that holds for every one of the 20 seeds. (iii) Laplace smoothing ($\alpha = 1$) has negligible influence at this sample size, there are no latent variables (so no initialisation issue), and inference is O(d). The dependence can be detected from the training data with the conditional mutual information I(Xi ; Xj | Y):

Pair	**(1,4)**	**(1,2)**	**(2,4)**	**(all other pairs)**
I(Xi; Xj | Y) in nats	**0.573**	0.0022	0.0022	≤ 0.0011

The duplicate pair has a CMI of 0.573 nats (close to the maximum for a binary variable, ln 2 = 0.693), about 250 times the next largest value, which is at the level expected from sampling noise (≈ df/(2N) = 2/2000 = 0.001 nats). This pinpoints a precise, detectable violation of the Naïve Bayes assumption.

B. Define the Design Objective

Item	Specification	Rationale
Baseline	Naïve Bayes with the duplicated feature: log-loss 0.6044, ECE 0.1438, accuracy 0.7468	Results of Q3 (same data generator, seeds 100–119)
Target	Log-loss ≤ 0.53 (within 3 % of the redundancy-free reference 0.5156) and ECE ≤ 0.04	Recovers the calibration that the model has when no feature is redundant
Constraints (acceptable trade-offs)	Accuracy not lower than baseline − 0.5 pp (≥ 0.742); no harm on data without redundancy (log-loss increase ≤ 0.005); training+prediction time up to 3 × baseline; extra memory for one 4-valued group table	Calibration must not be bought with accuracy or with a heavy model

C. Design the Revised Learning Strategy

What changes and why. Rather than adopting a more complex model wholesale, only the dependent pair is repaired. The pair (Xi, Xj) with a significant conditional mutual information is merged into one group variable G = (Xi, Xj) with four joint values, and Naïve Bayes is applied to the groups. This is a belief network (Q4) in which Y is the parent of each group and the independence assumption is relaxed only where the data show a violation: $P(Y \mid x) \propto P(Y) \cdot P(G1 \mid Y) \cdot P(X2 \mid Y) \cdot P(X3 \mid Y)$. It addresses the diagnosed property – double counting of dependent evidence – directly: the joint table $P(X1, X4 \mid Y)$ places all its mass on the agreeing cells (0,0) and (1,1), so the evidence is counted once.

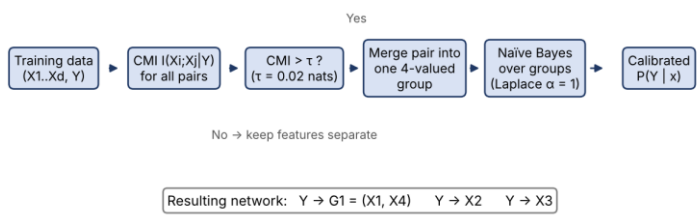


Figure Q7.1 – Workflow and resulting network structure

Pseudocode.

1. For every pair (i, j): compute I(Xi; Xj | Y) = sum_c P(c) sum_{a,b} P(a,b|c) ln[P(a,b|c) / (P(a|c) P(b|c))]
2. Sort pairs by CMI (descending). For each pair with CMI > tau (tau = 0.02 nats):
   if neither feature is already in a group: create group G = {Xi, Xj}
3. Every remaining feature forms its own group.
4. Fit: P(Y) and, for each group g, P(g = v | Y) with Laplace smoothing (alpha = 1; 2^|g| values).
5. Predict: log P(Y|x) = log P(Y) + sum_g log P(g(x) | Y); normalise with log-sum-exp.

Why $\tau = 0.02$ nats. For independent binary features the sampling noise of the CMI estimate is about df/(2N) = 0.001 nats at N = 1,000, so τ is about twenty times the noise level, but still ten times smaller than the CMI of a clearly redundant pair (0.573). It scales as 1/N, so it should be increased for smaller samples.

Additional computation, data and memory. CMI for all $d(d-1)/2$ pairs costs $O(d^2 N)$; each merged group needs 2×4 table entries instead of $2 \times 2 + 2 \times 2$ (a few extra parameters). **New limitations and trade-offs:** (i) a merged 4-cell table needs more data to estimate than two 2-cell tables (risk at small N); (ii) spurious merges when CMI estimates are noisy; (iii) only pairwise (not higher-order or chain) dependence is handled; (iv) a threshold hyper-parameter must be chosen; (v) CMI detects dependence given Y but does not say whether the dependence is harmful.

D. Predict Before Implementing

- **Main prediction:** on the duplicated-feature data (n = 1,000), log-loss falls from 0.604 to about 0.516 (within ± 0.01 of the redundancy-free value) and ECE falls from 0.144 to below 0.03.
- **Accuracy** stays within ± 0.5 percentage points of the baseline (0.7468), since accuracy was not damaged by the duplicate.
- The CMI screening merges the pair (X1, X4) in every run and merges nothing on data without redundancy, so results on clean data equal Naïve Bayes.
- For a 90 %-correlated (noisy) copy of X1 the improvement should be smaller than for the exact duplicate but still clearly positive.
- At a small sample (n = 100) the gain should shrink and the CMI test may produce occasional spurious merges, so plain Naïve Bayes may be slightly better on clean small data.

E. Implement and Validate the Redesign

Protocol. Same generator and seeds as Q3 (default_rng(100 + s), s = 0…19), 5,000 test records per seed, $\alpha = 1$, $\tau = 0.02$. Scenarios: exact duplicate X4 = X1, noisy copy (X4 agrees with X1 in 90 % of records), no redundancy, and small samples (n = 100). "Drop" is an oracle baseline that deletes the known redundant column (not available in practice). Values are means ± standard deviation over 20 seeds.

Scenario	Model	Accuracy	Log-loss	ECE	Merge in
Duplicate, n = 1,000	Naïve Bayes	0.7468 ± 0.0055	0.6044 ± 0.0255	0.1438	–
	CMI-merge	0.7468 ± 0.0057	**0.5156 ± 0.0057**	**0.0184**	20 / 20
	Drop (oracle)	0.7468 ± 0.0057	0.5156 ± 0.0057	0.0184	–
90 % copy, n = 1,000	Naïve Bayes	0.7465 ± 0.0068	0.5690 ± 0.0158	0.1102	–
	CMI-merge	0.7484 ± 0.0061	0.5158 ± 0.0068	0.0195	20 / 20
	Drop (oracle)	0.7494 ± 0.0059	0.5151 ± 0.0064	0.0184	–
No redundancy, n = 1,000	Naïve Bayes	0.7468 ± 0.0057	0.5156 ± 0.0057	0.0184	–
	CMI-merge	0.7468 ± 0.0057	0.5156 ± 0.0057	0.0184	0 / 20
Duplicate, n = 100	Naïve Bayes	0.7502 ± 0.0058	0.6204 ± 0.0846	0.1292	–
	CMI-merge	0.7435 ± 0.0166	0.5349 ± 0.0152	0.0621	20 / 20
	Drop (oracle)	0.7428 ± 0.0164	0.5314 ± 0.0132	0.0583	–
No redundancy, n = 100	Naïve Bayes	0.7428 ± 0.0164	0.5314 ± 0.0132	0.0583	–
	CMI-merge	0.7400 ± 0.0186	0.5374 ± 0.0160	0.0662	9 / 20 (spurious)

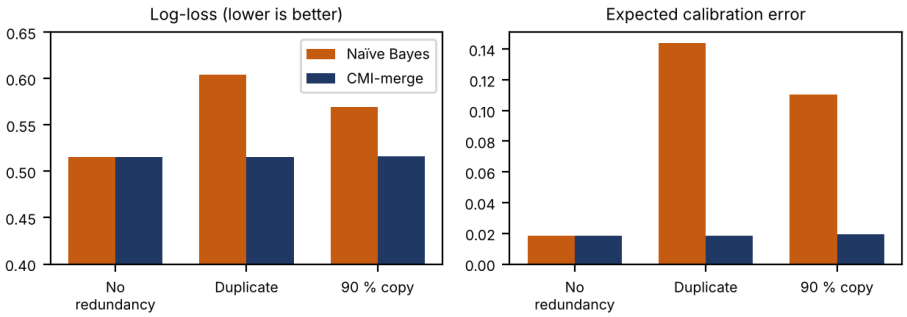


Figure Q7.2 – Log-loss and ECE of Naïve Bayes versus CMI-merge (n = 1,000, 20 seeds)

Paired comparison (duplicate, n = 1,000): the log-loss improved in all 20 seeds, mean gain 0.0888 (smallest gain 0.0611). **Cost:** fit + predict time per run rose from about 1.2 ms to 2.3–2.8 ms (≈ 2 ×) on a 1,000-record training set.

```
def cmi(a, b, y):          # I(A;B|Y) in nats from counts
    tot = 0.0
    for c in (0, 1):
        m = y == c; w = m.mean()
        if w == 0: continue
        pj = np.array([[((a[m] == i) & (b[m] == j)).mean() for j in (0, 1)] for i in (0, 1)]); pa = pj.sum(1, keepdims=True); pb = pj.sum(0, keepdims=True)
        with np.errstate(divide="ignore", invalid="ignore"): t = np.where(pj > 0, pj*np.log(pj/(pa*pb)), 0.0)
        tot += w*t.sum()
    return tot


def build_groups(X, y, thr):
    d = X.shape[1]; free = set(range(d)); groups = []
    pairs = sorted(((cmi(X[:, i], X[:, j], y), i, j) for i, j in itertools.combinations(range(d), 2)), reverse=True)
    for s, i, j in pairs:
        if s > thr and i in free and j in free: groups.append((i, j)); free -= {i, j}
    return groups + [(i,) for i in sorted(free)]


def fit_groups(X, y, groups, a=1.0):
    pri = np.array([(y == c).mean() for c in (0, 1)]); tabs = []
    for g in groups:
```

```
    K = 2**len(g); z = encode(X, g); tabs.append(np.array([[((z[y == c] == v).sum()+a)/((y == c).sum()+a*K) for v in range(K)] for c in (0, 1)]))
  return pri, tabs

def predict(m, X, groups):
  pri, tabs = m; ll = np.tile(np.log(pri), (len(X), 1))
  for g, t in zip(groups, tabs): ll += np.log(t[:, encode(X, g)]).T
  ll -= ll.max(1, keepdims=True); p = np.exp(ll); return p/p.sum(1, keepdims=True)
```

F. Justify the Final Design

Objective	**Target**	**Achieved (duplicate, n = 1,000)**	**Met?**
Log-loss	≤ 0.53	0.5156	Yes
ECE	≤ 0.04	0.0184	Yes
Accuracy	≥ 0.742	0.7468	Yes
No harm on clean data (n = 1,000)	≤ +0.005 log-loss	0.0000 (no merges)	Yes
Time	≤ 3 × baseline	≈ 2 ×	Yes

Prediction versus outcome. The main prediction was confirmed: log-loss fell to 0.5156 (equal to the redundancy-free model) and ECE to 0.0184; accuracy was unchanged; the pair was merged in 20/20 runs and never merged on clean data at n = 1,000. The noisy-copy prediction was also confirmed: the gain was smaller than for the exact duplicate (0.569 → 0.516, a drop of 0.053 versus 0.089) because the baseline damage is smaller, yet the merged model again matched the redundancy-free log-loss (0.5158 versus 0.5156). At n = 100 the gain persisted (0.620 → 0.535) but was reduced, and, as predicted, the CMI test produced spurious merges on clean data.

Costs and limitations. About twice the computation (still milliseconds), a threshold hyper-parameter τ that depends on N, a larger table for each merged pair, and only pairwise dependence is handled. For an exact duplicate, simply dropping the column gives identical results more cheaply (0.5156 / 0.0184), and in the noisy case dropping was marginally better (0.5151 vs 0.5158, within one standard deviation); merging is preferable when the dependence is real but the second feature still carries some independent information, or when we cannot know which member of the pair to drop.

When the original Naïve Bayes remains preferable. (i) When only the class decision matters: accuracy was never affected by the redundancy. (ii) When features are (nearly) independent and the sample is very small: at n = 100 on clean data plain NB had log-loss 0.5314 versus 0.5374 for CMI-merge because noisy CMI estimates caused spurious merges in 9 of 20 runs. (iii) When simplicity or speed outweigh calibration.

Final design decision. Adopt Naïve Bayes with CMI-guided merging ($\tau = 0.02$ nats at $N \approx 1{,}000$, scaled as $1/N$) whenever the predicted probabilities are used for decisions (risk, cost, thresholds), keeping plain Naïve Bayes as the fallback for very small samples ($N \lesssim 100$) or when only labels are needed. The redesign meets every measurable objective, is justified by the CMI diagnosis, and adds only modest computation.